

WATCH SYNTAX AND THINK LOGICALLY [.item and .data are used to grab values along with .getNode()]

Prev = reference to the previous node; *Next* = reference to the next node; *head* = first node; *tail* = last node

Node<E> adding = new node<>(); LinkedList<Integer> out = new LinkedList<>();

Remove tail of a DOUBLY LinkedList

```
tail = tail.prev;          for (Type (String) variable (word) : list){
```

```
tail.next = null;          String.length() , ArrayList.size()
```

```
Size--;                    **In General runtime can be calculated by counting what for loops iterate over**
```

Add a new item to index 0 of a doubly linked list

```
reverseWords(){
```

```
Node<E> adding = new node<>();
```

```
for(String word: list){
```

```
adding.next = head;
```

```
String reversed = "";
```

```
head.prev = adding;
```

```
for(char c : word.toCharArray()){
```

```
head = adding;
```

```
reversed = c + reversed;
```

```
size++;
```

```
}
```

```
Output.add(reversed);
```

```
deleteList(){
```

DOUBLY LINKED LIST

```
while(head!=null){
```

```
    head.prev = null;
```

```
    head = head.next;
```

```
}
```

Singly Linked List

```
tail = null;
```

```
size = 0;
```

```
Node<E> before = getNode(index - 1)
```

****Singly Linked list should use for each loop or .get(i)****



Reverse a doubly linked list → goal is to swap next and prev; store prev in a temp variable set prev = next then next = temp (stored prev) → Move nodes via prev since they swapped and keep switching → find the new head (when prev == null) and then make that the new head and set the pointer to that head

With an **array list** there is potentially not used space and if you run out of space you will have to reallocate the list and that usually means you double it and could mean $O(n)$ extra slots; Constant time to add or remove to the head or middle and removing the end but removing the end can be $O(n)$ in the case you have to reallocate; Get and set are constant time because they use arrays and you get there instantly via an index

With a **LinkedList** there is $O(n)$ extra space as well; Constant time $O(1)$ to add or remove to the head or tail but $O(n)$ for anything else; Get and set are $O(n)$ because you need to go through each element to get to the point you need to be at; In general, LinkedList are better to add and remove from

Constant time $\rightarrow O(1)$, Linear time $\rightarrow O(n)$, Quadratic time $\rightarrow O(n^2)$

```
Public void add(E item){
    Node<E> adding = new Node<>();
    adding.item = item;
    if (size == 0){
        head = adding;
        tail = adding;
    } else if (item <= head.item){
// this is for adding at head if it is tail then you do >

        Adding.next = head;
        Head = adding;
    } // adding anywhere else you check if its less
// than or equal to the next node so you are now it's in the correct spot // SAME FOR LIST B THEN RETURN
// then set adding.next = current.next and current.next = adding
```

```
public static LinkedList<Integer> merge(){
    LinkedList<Integer> out = new LinkedList<>();
    while(listA.size() > 0 && listB.size() > 0){
        if(listA.get(0) < listB.get(0)){
            out.add(listA.remove(0));
        } else {
            out.add(listB.remove(0));
        }
    }
    while(listA.size() > 0){
        out.add(ListA.remove(0)); }
}
```

```
Public int minPlusMax(List <Integer> list){
// This is a function to find the lowest/Highest value

    Int min = Integer.MAX_VALUE;
    Int max = Integer.MIN_VALUE;
```

**For adding and removing nodes think:
index out of bounds, empty list, adding head,
adding tail, anything else**

```
For(int I = 0; I < list.size(); i++){
    Int num = list.get(i);
    If (num < min) {
        Min = num;
    }
    If (num > max){
        Max = num;
    }
}
Return min + max; } // think about how to reassign these values write the notes with each command
```

```
Node<E> adding = new Node(item);
if(size == 0) { // if empty list
    head = adding;
    tail = adding;
    tail.next = head;
} else if (index == 0){ // adding at the head
    adding.next = head; // new node points to old head
    head = adding; // new node becomes head
    tail.next = head; // tail now points to the new head
```